

MINI-KONCILI

PLANO TÉCNICO DO PROJETO

MVP em 12 dias — sistema de conciliação de vendas e repasses,
inspirado no Konkili (DB1 Group)

Backend: Laravel + Livewire
Frontend: Blade + Tailwind CSS
Animações: GSAP + Alpine.js

Este documento responde, ponto a ponto, cada pergunta técnica levantada: quantas páginas o sistema deve ter, o que animar, arquitetura, banco de dados, cache, autenticação, documentação de API, uso do GitLab e a página bônus "Sobre mim" — fechando com o cronograma dia a dia.

Visão geral

O Mini-Koncili simula, em escala pequena, o que o produto real Konkili (da DB1) faz: cruzar vendas registradas em marketplaces com os repasses financeiros efetivamente recebidos, identificando divergências de valor, comissão ou repasses ausentes. A lógica de conciliação usada no mercado real (confirmada em pesquisa) é conciliar **por código do pedido**, comparando o valor esperado (venda – comissão – taxas) com o valor efetivamente repassado — essa é exatamente a lógica que o projeto implementa.

O objetivo não é competir com o Konkili — é ter, em 12 dias, uma versão pequena, honesta e funcional, que prove que você entende o problema de negócio que a DB1 resolve, com uma stack moderna e completa (Livewire, cache, API documentada, autenticação segura).

Ambiente de desenvolvimento

Já está pronto e não entra no cronograma dos 12 dias — só o registro do que está definido:

- Docker Compose já montado, com containers separados: apache, mysql, nginx, node, php-fpm, proxy, redis, ssl
- Servidor: Ubuntu Server rodando em VM, com o VS Code conectado remotamente (Remote-SSH)
- Redis já disponível — usado desde o início para o cache do Dashboard, não como extra

Isso encolhe o Dia 1 do cronograma: não é preciso montar ambiente, só entrar direto em modelagem.

Paleta de cores (inspirada na DB1)

Aviso importante: não consegui extrair o hex exato do site oficial da DB1 (isso exige inspecionar o CSS deles diretamente). O que confirmei por pesquisa é que a marca passou por um rebranding onde "se tornou azul para enfatizar o sentido de tecnologia", com um resquício de vermelho da identidade antiga como acento histórico. A paleta abaixo é uma base coerente com isso — não uma cópia oficial pixel-a-pixel. Para precisão total, abra o site da DB1 → F12 → Elements → confira o hex do header.

Uso	Cor	Hex
Fundo escuro / navbar / headers	Azul-marinho profundo	#0B2545
Cor primária (botões, links, destaques)	Azul DB1	#1857A4
Fundo claro / cards / hover	Azul muito claro	#EAF1FB
Texto principal	Quase-preto	#161616
Texto secundário	Cinza	#5A5A5A
Status: conciliado	Verde	#2E7D32
Status: divergente	Âmbar	#C77700
Fundo geral	Branco	#FFFFFF

Quantas páginas o sistema deve ter

7 páginas funcionais + 1 página bônus. Esse número é o ponto de equilíbrio: dá pra mostrar um fluxo completo (login → dados → conciliação → relatório) sem estourar os 12 dias com telas que não agregam ao argumento central.

- 1. Login / Registro** — Autenticação via Laravel Breeze (stack Livewire).
 - 2. Dashboard** — Métricas gerais: total conciliado, total divergente, % conciliado no mês, gráfico simples. Dados em cache.
 - 3. Vendas** — Listagem + cadastro de vendas (CRUD via Livewire), com busca e paginação.
 - 4. Repasses** — Listagem + cadastro de repasses recebidos (CRUD via Livewire).
 - 5. Conciliação** — Tela principal: cruza vendas x repasses e mostra status (conciliado / divergente / pendente). O coração do projeto.
 - 6. Relatório** — Exportação simples (CSV) das divergências encontradas no período.
 - 7. Sobre o projeto** — Explica o propósito do projeto — por que foi feito, inspiração no Konkili.
- Bônus — Sobre mim** — Página curta com seu resumo profissional (ver seção específica).

Página de "Perfil/Configurações" fica de fora do escopo principal — é a primeira coisa a cortar se o tempo apertar, porque não agrega ao argumento de negócio.

O que deve ser animado (GSAP + Alpine.js)

Regra geral: animação discreta e funcional, não decorativa. É um sistema financeiro — exagerar nas animações passa a impressão errada. Cada uma abaixo tem um motivo prático.

Elemento	Ferramenta	Por quê
Contadores do Dashboard	GSAP	Números (total conciliado, divergente) sobem de 0 até o valor real ao carregar — reforça a sensação de "dado vivo".
Barra de % conciliado	GSAP	Barra de progresso anima até a % real do mês.
Entrada dos cards	GSAP (fade + slide-up leve)	Cards do dashboard aparecem com leve atraso entre si (stagger) — sensação de painel profissional, não de tabela estática.
Estados de carregamento	Alpine.js (wire:loading)	Enquanto o Livewire processa (ex: rodar a conciliação), mostra spinner/skeleton — evita sensação de tela travada.
Badges de status	Alpine.js (transition)	Quando o status de uma linha muda (pendente → conciliado), a cor do badge transiciona suavemente.
Modais (novo lançamento, detalhes de divergência)	Alpine.js (x-transition)	Abrir/fechar com fade + scale leve, padrão do ecossistema Livewire + Alpine.

Perguntas técnicas, respondidas direto

O site se conecta a um banco de dados de verdade?

- Sim. MySQL, com migrations reais (nada de dado mockado direto no código) — mesmo banco que você já domina nos outros projetos.
- 4 tabelas principais: users, sales (vendas), transfers (repasses) e reconciliations (conciliações).

Usa cache?

- Sim, nos números agregados do Dashboard (total conciliado, total divergente, % do mês) — são cálculos que não precisam ser refeitos a cada clique.
- `Cache::remember()` com TTL de ~10 minutos, usando Redis (você já tem o container pronto no Docker — usar desde o início, não como extra de última hora).
- Invalidação automática: um Model Observer limpa o cache sempre que uma venda, repasse ou conciliação é criada/alterada — assim o painel nunca mostra número desatualizado.

Tem autenticação segura?

- Sim. Laravel Breeze no stack Livewire cuida de: hash de senha (bcrypt), proteção CSRF automática, rate limiting no login (throttle, evita força bruta).
- Cada usuário só vê os próprios dados (Policy do Laravel) — evita que um usuário veja a conciliação de outro.
- Para a API: autenticação via token (Laravel Sanctum), separada do login da tela.

Como está a arquitetura do projeto?

- Camada de dados: Models Eloquent (Sale, Transfer, Reconciliation) + migrations.
- Camada de regra de negócio: uma classe de serviço, `ReconciliationService`, isolada dos componentes visuais — é ela que faz o "match" entre vendas e repasses. Isso separa "lógica de conciliação" de "tela", o que é boa prática e fácil de explicar numa entrevista.
- Camada de interface: componentes Livewire (`SalesTable`, `TransfersTable`, `ReconciliationPanel`, `Dashboard`) — PHP e interatividade no mesmo lugar, sem precisar escrever uma API separada só pra tela funcionar.
- Camada de API: Controllers REST + API Resources, funcionando em paralelo à interface Livewire, para quem quiser consumir os dados programaticamente.

Esquema simplificado das camadas:

```
Blade + Tailwind (visual)
|
Livewire Components (interatividade: SalesTable, ReconciliationPanel...)
|
ReconciliationService (regra de negocio: faz o match venda x repasse)
|
Eloquent Models (Sale, Transfer, Reconciliation, User)
|
MySQL

API REST (Sanctum) -> mesmos Models, acesso paralelo via /api/v1/...
```

A lógica de conciliação (o coração do projeto)

Confirmado por pesquisa de mercado: sistemas reais de conciliação (incluindo o próprio Konkili) cruzam **por código do pedido**, comparando o valor esperado (venda – comissão – taxas) com o valor efetivamente repassado. O ReconciliationService replica essa lógica:

- Para cada venda, procura um repasse com o mesmo código de pedido (order_code)
- Se encontrar e o valor bater (dentro de uma margem de centavos): status = **conciliado**
- Se encontrar mas o valor divergir: status = **divergente**, com o valor da diferença calculado e salvo
- Se não encontrar repasse nenhum: status = **pendente**

Esse é exatamente o tipo de explicação que vale ouro numa conversa técnica — mostra que você não só copiou uma ideia, mas entendeu o problema real por trás dela.

Documentação da API

Com 12 dias, não vale a pena aprender uma ferramenta nova de documentação do zero (tipo Swagger/OpenAPI) — o retorno não compensa o tempo. Caminho recomendado, usando o que você já sabe:

- **Coleção no Insomnia/Postman** (você já usa) com todos os endpoints organizados por pasta: Auth, Sales, Transfers, Reconciliations — exportar o arquivo .json e deixar no repositório
- **Um arquivo API.md** no repositório, simples e direto: método, rota, parâmetros, exemplo de resposta JSON para cada endpoint
- Endpoints principais: POST /api/v1/login · GET/POST /api/v1/sales · GET/POST /api/v1/transfers · GET /api/v1/reconciliations

Isso entrega uma API documentada de verdade, sem gastar 1 dos 12 dias aprendendo Swagger.

Sobre usar o GitLab

Pesquisei vagas reais da DB1 (backend, frontend, .NET, Golang, Python) e não encontrei nenhuma menção a GitLab — todo o material público que achei aponta pra um fluxo comum de Git, sem confirmar qual plataforma usam internamente. Ou seja: **não consegui confirmar essa informação**, então é melhor não apostar tudo nisso.

Dito isso, espelhar o repositório no GitLab é barato — literalmente 2 comandos (git remote add gitlab ... / git push gitlab main) — e não atrapalha em nada. Recomendo fazer isso só no último dia (dia 12), como um extra de 10 minutos, não como parte do planejamento principal. Se em algum momento você confirmar com um recrutador que a DB1 usa GitLab internamente, aí sim vale mencionar ativamente isso na conversa.

Alpine.js e Livewire: até onde estudar

Ponto de partida honesto: o motivo do front-end "parecer morto" quase nunca é falta de saber Alpine — é não usar os ganchos de transição que o próprio Livewire já oferece pronto.

- **Alpine.js:** é pequeno, dá pra dominar o essencial em 1-2h de tutorial direto — `x-data`, `x-show`, `x-transition`, `x-on`, `x-model`. Não precisa de curso longo.
- **Livewire (o que realmente dá vida):** `wire:loading` e `wire:target` (estados de carregamento), `wire:transition` (Livewire 3 já traz transições nativas), e `Alpine.entangle()` pra Alpine "escutar" o estado do componente Livewire.
- Todo o resto (diretivas menos comuns, plugins de Alpine) fica no modo "pesquisar quando precisar", não estudo dedicado.

Não vale estudar a fundo (gasta tempo dos 12 dias sem ganho proporcional) nem deixar de usar (joga fora a decisão de stack já tomada).

Uso de IA no desenvolvimento — disciplina

A própria DB1 divulga publicamente um case interno de adoção de Copilots de IA nos times de desenvolvimento: +23% em entregas, -70% em dívida técnica, com adoção "gradual e orientada". Ou seja: usar IA para construir esse projeto está alinhado com a prática real da empresa — não é algo a esconder.

Regra prática: IA pode acelerar, nunca pode substituir seu entendimento.

- **Pode usar sem culpa:** boilerplate (esqueleto de migration, CRUD básico), debugar mensagem de erro desconhecida, explicar um padrão novo do Alpine/Livewire
- **Use com revisão linha por linha:** componentes maiores (ex: o ReconciliationPanel inteiro) — bom ponto de partida, mas reescreva trechos com a própria mão
- **Não terceirize:** a lógica do ReconciliationService — é exatamente o que vão perguntar a fundo numa entrevista técnica

Teste de disciplina antes de seguir pro próximo passo: "eu consigo fechar o editor agora e reescrever essa lógica com minhas palavras, sem olhar?" Se não, pare e estude aquele trecho antes de continuar. E não tem problema nenhum ser transparente sobre isso numa entrevista — dado que a própria DB1 trata IA como parte do processo.

Testes automatizados (Pest)

"Testes automatizados" aparece como atributo valorizado em vagas reais da DB1 que pesquisei — vale reservar um espaço curto pra isso, mesmo em 12 dias. Como você já sabe testar API com Pest, não precisa de tempo de aprendizado aqui, só de execução.

- 2-3 testes cobrindo só o ReconciliationService: caso conciliado, caso divergente, caso pendente
- Não precisa testar o projeto inteiro — só o coração da lógica de negócio

Página bônus: Sobre mim

Uma rota simples dentro do próprio sistema (ex: /sobre), sem precisar de login pra ver — assim quem receber o link acessa direto. Sugestão de conteúdo, curto e objetivo:

- Nome, formação (Tecnólogo em Sistemas para Internet — UNIALFA) e localização (Umuarama, PR — Home Office)
- 3-4 linhas explicando por que esse projeto existe: entender na prática o problema de conciliação financeira que empresas como a DB1 resolvem
- Stack usada no projeto (Laravel, Livewire, Tailwind, GSAP, Alpine.js) — mostra que você sabe nomear o que construiu
- Links: LinkedIn, GitHub, portfólio (wagner-dev.free.nf)

Isso transforma o próprio projeto em cartão de visitas — quem for avaliar não precisa sair do link para saber quem construiu e por quê.

Simulação: o que um RH perguntaria

Comportamentais

- Me conta um pouco sobre você e sua trajetória até aqui
- Por que você quer trabalhar na DB1, especificamente?
- Como você concilia a faculdade à noite com um emprego CLT?
- Me conta de um projeto que você tem orgulho — por quê?
- Como você lida com prazo apertado ou algo dando errado na última hora?
- Você prefere trabalhar mais sozinho ou em time?

Técnicas-leves (RH testa clareza, não profundidade)

- Explica esse projeto pra mim como se eu não fosse da área
- Por que Laravel, e não outra tecnologia?
- O que foi mais difícil de resolver nesse projeto?
- O que esse projeto tem a ver com o que a DB1 faz?

Cronograma — 12 dias

Substitui o bloco do habit-tracker no seu plano de estudos original (6 a 19/ago). Mesma lógica: blocos curtos em dia de semana, blocos longos no fim de semana.

FASE 1 — Fundação (dias 1 a 4)

Sáb 8/ago a Ter 11/ago

DIA 1 Sáb
8/ago

Modelagem completa

3h

- Ambiente já está pronto (Docker) — pula setup, vai direto pra modelagem
- Desenhar as 4 tabelas (users, sales, transfers, reconciliations), criar migrations e seeders com dados de teste

DIA 2 Dom
9/ago

Autenticação + layout base

3-4h

- Login/registro funcionando (Breeze Livewire)
- Layout base com Tailwind + navegação com Alpine.js (menu, dropdown)

DIA 3 Seg
10/ago

CRUD de Vendas

45min

- Componente Livewire SalesTable: listar, criar, editar, buscar

DIA 4 Ter
11/ago

CRUD de Repasses

45min

- Componente Livewire TransfersTable, mesmo padrão do dia anterior

FASE 2 — O coração do projeto (dias 5 a 8)

Qua 12/ago a Sáb 15/ago

DIA 5 Qua
12/ago

ReconciliationService + testes Pest

1h

- Classe de serviço com a lógica de match por código do pedido
- 2-3 testes Pest cobrindo os 3 cenários (conciliado, divergente, pendente)

DIA 6 Qui
13/ago

Tela de Conciliação

45min

- Componente Livewire ReconciliationPanel: mostra o cruzamento com status coloridos
- Filtros por status (conciliado / divergente / pendente)

DIA 7 Sex
14/ago

Interatividade (Alpine + Livewire)

1h

- wire:loading / wire:transition nos estados de carregamento
- Alpine.entangle() nos badges de status e modais

DIA 8 Sáb
15/ago

Dashboard + cache Redis + GSAP

3-4h

- Métricas agregadas com Cache::remember() (Redis) + Observer de invalidação
- Contadores animados e barra de progresso com GSAP
- Stagger de entrada dos cards

FASE 3 — API, documentação e polish (dias 9 a 12)

Dom 16/ago a Qua 19/ago

DIA 9
Dom
16/ago

API REST 3h

- Controllers /api/v1 (sales, transfers, reconciliations) + Sanctum
- API Resources para formatar o JSON de resposta

DIA 10
Seg
17/ago

Documentação da API 1h

- Montar a coleção no Insomnia/Postman
- Escrever o API.md no repositório
- Se sobrar tempo: botão opcional "Explicar divergência" com IA (último item da lista, primeiro a cortar)

DIA 11
Ter
18/ago

Página Sobre mim + revisão 1h

- Rota /sobre com o conteúdo definido acima
- Revisar responsividade e transições
- Passar pelo teste de disciplina: revisar tudo que a IA ajudou a gerar e confirmar que consegue explicar cada trecho

DIA 12
Qua
19/ago

Deploy + retoques finais 1-2h

- Deploy em produção (hospedagem simples, igual o Epic Fight Figures)
- README completo do projeto no GitHub
- Espelhar o repositório no GitLab (opcional, 10min — ver seção sobre isso)
- Gravar vídeo curto de backup, seguindo o plano B já combinado pra imprevistos técnicos

Se algum dia atrasar, corte primeiro (nessa ordem): página de Perfil (já fora do escopo), exportação CSV do Relatório, espelho no GitLab. Nunca corte a lógica de conciliação — é o argumento central do projeto.